

C Programming – Comprehensive Syllabus

Course Overview

This course provides a strong foundation in C programming, covering fundamental programming concepts, problem-solving techniques, data structures, file handling, and modular programming. Students will develop logical thinking and coding skills through practical exercises and real-world applications.

Duration: 8–12 Weeks (50–70 Hours)

Prerequisites:

- Basic computer knowledge
 - No prior programming experience required
-

Module 1: Introduction to C Programming (Week 1)

Topics

- Introduction to Programming
- History and Features of C
- Installing C Compiler and IDE
- Structure of a C Program
- Keywords and Identifiers
- Variables and Constants
- Data Types
- Input and Output Functions
- Comments

Practical Exercises

- Hello World Program
 - Basic Arithmetic Calculator
 - Unit Conversion Programs
-

Module 2: Operators and Control Statements (Week 2)

Topics

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Bitwise Operators
- Conditional Operator
- if, if-else, Nested if
- switch Statement

Practical Exercises

- Grade Calculator
 - Simple Calculator
 - Leap Year Checker
 - Menu-Driven Programs
-

Module 3: Loops and Functions (Week 3)

Topics

- for Loop
- while Loop
- do-while Loop
- break and continue
- Functions
- Function Declaration and Definition
- Function Arguments
- Return Values
- Recursive Functions
- Scope of Variables
- Storage Classes

Practical Exercises

- Factorial Program
- Fibonacci Series
- Prime Number Checker

- Recursive Functions
-

Module 4: Arrays and Strings (Week 4)

Topics

- One-Dimensional Arrays
- Two-Dimensional Arrays
- Multidimensional Arrays
- String Handling
- Standard String Functions
- Character Arrays

Practical Exercises

- Matrix Addition
 - Matrix Multiplication
 - String Reversal
 - Palindrome Checker
-

Module 5: Pointers (Week 5)

Topics

- Introduction to Pointers
- Pointer Arithmetic
- Pointers and Arrays
- Pointers to Functions
- Dynamic Memory Allocation
- malloc(), calloc(), realloc(), free()

Practical Exercises

- Array Operations Using Pointers
 - Dynamic Memory Allocation Programs
 - Pointer-Based Swapping
-

Module 6: Structures, Unions & Enumerations (Week 6)

Topics

- Structures
- Nested Structures
- Arrays of Structures
- Pointers to Structures
- Unions
- Enumerations (enum)
- typedef

Practical Exercises

- Student Record Management
 - Employee Database
 - Inventory Management
-

Module 7: File Handling & Preprocessor Directives (Week 7)

Topics

- File Operations
- Text Files
- Binary Files
- File Modes
- Reading and Writing Files
- Command Line Arguments
- Preprocessor Directives
- Macros
- Header Files

Practical Exercises

- Student Data File
 - File Copy Utility
 - Report Generation
-

Module 8: Data Structures Using C (Week 8)

Topics

- Introduction to Data Structures
- Linked Lists
- Stacks
- Queues
- Trees (Basics)
- Searching Algorithms
- Sorting Algorithms

Practical Exercises

- Stack Implementation
 - Queue Implementation
 - Linked List Operations
 - Sorting and Searching Programs
-

Module 9: Mini Project (Week 9)

Choose one:

- Student Management System
 - Library Management System
 - Banking System
 - Employee Payroll System
 - Inventory Management System
 - Hotel Management System
 - Online Quiz Application (Console-Based)
-

Module 10–12: Capstone Project

Complete Software Development Project

- Problem Analysis
- Algorithm Design
- Flowchart Development

- Program Design
 - Modular Coding
 - Testing and Debugging
 - Documentation
 - Final Demonstration
-

Tools & Software

- GCC Compiler
 - Turbo C (for learning purposes)
 - Code::Blocks
 - Dev-C++
 - Visual Studio Code
 - CLion (Optional)
 - Git & GitHub
-

Core Concepts Covered

- Programming Fundamentals
 - Problem Solving
 - Algorithms and Flowcharts
 - Functions and Modular Programming
 - Arrays and Strings
 - Pointers
 - Dynamic Memory Allocation
 - Structures and Unions
 - File Handling
 - Data Structures
 - Searching and Sorting Techniques
-

Assessment Scheme

Component	Weightage
Assignments	20%
Lab Exercises	20%

Quizzes	10%
Mini Project	20%
Capstone Project	30%

Learning Outcomes

By the end of the course, learners will be able to:

- Understand the fundamentals of C programming and structured programming concepts.
- Develop algorithms and implement efficient solutions using C.
- Use decision-making, loops, functions, arrays, strings, and pointers effectively.
- Apply dynamic memory allocation and manage memory efficiently.
- Design programs using structures, unions, and modular programming techniques.
- Perform file handling operations for persistent data storage.
- Implement basic data structures and searching/sorting algorithms.
- Debug, test, and optimize C programs using industry-standard development tools.
- Build real-world console-based applications using best coding practices.